



Filière : Master en Informatique et Télécommunications

Développement d'un système intégré de reconnaissance de caractères imprimés et de traduction de texte utilisant OCR et GRNN

Réalisé par :

AGNAOU ABDELLAH REDA BAKKALI MERYEM EL MANSOUR

Encadré par :

Pr.Yassine Benjelloune Touimi

Année universitaire : 2022-2023

Résumé

Cette thèse se concentre sur le développement d'un système intégré de reconnaissance de caractères imprimés qui permet de convertir ce texte présenté par une image numérique en un texte modifiable, puis de procéder à sa traduction à travers une interface simple en utilisant l'OCR (Optical Character Recognition) et un réseau GRNN (Gated Recurrent Neural Network).

Les principales étapes entreprises dans cette recherche sont les suivantes :

1. Étape 1 : Étude de l'OCR pour la reconnaissance des caractères imprimés :

- Exploration des techniques d'OCR pour la conversion de l'image de texte imprimé en texte numérique.
- Utilisation de méthodes de prétraitement d'image pour améliorer la qualité de l'image et faciliter la reconnaissance des caractères.
- Mise en œuvre d'algorithmes d'OCR avancés pour détecter et interpréter les caractères imprimés avec une précision élevée.

2. Étape 2 : Utilisation d'un réseau GRNN pour la reconnaissance des caractères :

- Introduction du réseau GRNN, une architecture de réseau récurrente qui permet de prendre en compte la séquence des caractères dans le processus de reconnaissance.
- Entraînement du réseau GRNN sur des données de chiffres afin de capturer les motifs et les dépendances séquentielles des caractères.

3. Étape 3 : Intégration de la traduction du texte, de la synthèse vocale et mise en place d'une interface:

- Intégration d'un système de traduction automatique du texte reconnu dans différentes langues et d'un système de synthèse vocale.
- Développement d'une interface conviviale qui permet aux utilisateurs d'insérer une image contenant du texte, de visualiser les résultats de reconnaissance et de traduire instantanément le texte.
- Optimisation de l'interface pour offrir une expérience utilisateur agréable, intuitive et accessible à différents niveaux de compétence.

L'objectif de cette thèse est de créer un système intégré qui combine les avantages de l'OCR, du réseau GRNN, de la traduction de texte, de la synthèse vocale et d'une interface simple. Les résultats attendus incluent une amélioration de la précision de la reconnaissance des caractères grâce à l'utilisation du réseau GRNN, ainsi que la capacité de traduire instantanément le texte reconnu dans différentes langues.

Ce système intégré présente des applications potentielles dans divers domaines, tels que la numérisation de documents multilingues, la communication interlinguistique et la reconnaissance des plaques d'immatriculation. En exploitant les capacités de l'OCR, du réseau GRNN, de la traduction de texte et de l'interface conviviale, cette thèse vise à fournir une solution complète et pratique pour la reconnaissance, la traduction et l'utilisation efficace du texte imprimé.

TABLE DES MATIÈRES

1	Introduction générale	8
1.1	Problématique du projet	9
1.2	Organisation du rapport	10
2	Fondements théoriques et état de l'art dans la reconnaissance de caractères	11
2.1	Les caractères	11
2.1.1	Reconnaissance automatique des caractères	12
2.1.2	L'objectif de la reconnaissance des caractères	13
2.1.3	Les problèmes de la reconnaissance de caractères	14
2.1.4	Domaine d'application	15
2.2	La traduction de texte	16
2.3	Synthèse vocale	17
2.4	L'interface utilisateur (GUI)	18
3	Approche intégrée basée sur OCR et GRNN	20
3.1	Le modèle OCR	21
3.1.1	La fonctionnalité de l'OCR	22
3.2	Le réseau GRNN	24
3.2.1	Procédure d'entraînement	25
3.2.2	Avantages de GRNN	25
3.2.3	Inconvénients de GRNN	26
3.3	Conclusion	26

4	Outils et environnements de développement	27
4.1	Bibliothèques et outils utilisés dans l'implémentation	27
4.1.1	Python	27
4.1.2	PYTORCH	28
4.1.3	NumPy	30
4.1.4	Python-tesseract	31
4.1.5	gTTS (Google Text-to-Speech)	32
4.1.6	pyttsx3	33
4.1.7	Tkinter (Tool kit interface)	33
4.2	Base de données MNIST	34
5	Application et implémentation pratique	35
5.1	Acquisition de l'image	36
5.2	Prétraitement de l'image	36
5.3	Les paramètres principaux pour l'entraînement	37
5.3.1	batch_size et num_epochs	37
5.3.2	Mélange des données : shuffle	39
5.4	Le concept POO sur python	39
5.4.1	Classes	39
5.4.2	Objets	40
5.4.3	Attributs	40
5.4.4	Methodes	40
5.4.5	Encapsulation	40
5.4.6	Héritage	40
5.4.7	polymorphisme	41
5.5	Reconnaissance et classification des caractères	41
5.5.1	La construction du modèle GRNN	41
5.5.1.1	Le fonctionnement du modèle GRNN	41
5.5.1.2	Les optimiseurs	42
5.6	Traduction du texte	44

5.7	Synthèse vocale	44
5.8	Création de l'interface	45
6	Conclusion	47

Chapitre 1

Introduction générale

La reconnaissance des caractères imprimés est une tâche cruciale dans le traitement de texte automatisé, mais la capacité de traduire ces caractères dans différentes langues et de fournir une interface conviviale pour les utilisateurs est tout aussi importante. Cette thèse propose le développement d'un système intégré de reconnaissance de caractères imprimés, de traduction vocale de texte et d'interface conviviale, permettant une expérience utilisateur fluide et efficace.

Dans cette thèse, nous avons d'abord exploré les techniques avancées de reconnaissance de caractères imprimés, telles que la segmentation précise des caractères, l'extraction robuste des caractéristiques et la classification. En adoptant une approche basée sur le traitement d'image, la vision par ordinateur et les réseaux de neurones profonds, nous avons travaillé à améliorer la précision et la robustesse de la reconnaissance des caractères imprimés.

Ensuite, nous avons étendu notre système en intégrant une fonction de traduction automatique du texte reconnu dans deux langues. Cela permet aux utilisateurs d'obtenir rapidement une traduction vocale du texte imprimé dans leur langue préférée, facilitant ainsi la compréhension et la communication dans des contextes multilingues.

Enfin, nous avons développé une interface simple qui permet aux utilisateurs d'interagir facilement avec le système. L'interface est conçue de manière simple, offrant des fonctionnalités telles que l'importation de l'image contenant le texte, la visualisation des résultats de reconnaissance et la traduction vocale instantanée. Nous avons également pris en compte des aspects d'utilisabilité, en nous assurant que l'interface est simple et accessible aux utilisateurs de différents niveaux de compétence.

Ce système intégré de reconnaissance de caractères imprimés, de traduction vocale de texte et d'interface a de nombreuses applications potentielles, notamment dans les

domaines de la numérisation de documents multilingues, de l'aide à la communication interlinguistique et la reconnaissance des plaques d'immatriculation pour les utilisateurs ayant des compétences linguistiques limitées.

En conclusion, cette thèse vise à développer un système complet et polyvalent de reconnaissance de caractères imprimés et de traduction de texte à travers une interface. En combinant ces fonctionnalités, nous souhaitons offrir aux utilisateurs une solution pratique et efficace pour la reconnaissance et la traduction vocale de texte, tout en garantissant une expérience utilisateur agréable.

1.1 Problématique du projet

La problématique centrale de ce projet réside dans le développement d'un système intégré de reconnaissance de caractères robuste. Plus spécifiquement, les problématiques abordées sont les suivantes :

1. Précision de la reconnaissance des caractères imprimés : Comment améliorer la précision de la reconnaissance des caractères imprimés en utilisant des techniques avancées d'OCR et en exploitant les caractéristiques séquentielles des caractères à l'aide d'un réseau GRNN ?
2. Traduction automatique du texte reconnu : Comment intégrer un système de traduction automatique dans le processus de reconnaissance de caractères imprimés afin de permettre une traduction vocale instantanée du texte reconnu dans différentes langues ?
3. Création d'interface pour une expérience utilisateur optimale : Comment concevoir et mettre en place une interface qui facilite l'interaction entre les utilisateurs et le système, offrant des fonctionnalités intuitives telles que l'importation de l'image contenant le texte, la visualisation des résultats de reconnaissance et la traduction vocale instantanée ?

La résolution de ces problématiques est essentielle pour fournir un système intégré efficace et pratique, capable de reconnaître avec précision les caractères imprimés, de traduire le texte dans différentes langues et d'offrir une expérience utilisateur agréable. Ces éléments permettront de faciliter la transformation numérique des documents imprimés, d'améliorer la communication multilingue et d'accroître l'accessibilité à l'information pour les utilisateurs ayant des compétences linguistiques limitées.

1.2 Organisation du rapport

Dans cette recherche, nous nous intéressons aux étapes de l'extraction et de la reconnaissance de caractères. Dans ce contexte, nous proposons une méthode basée sur l'OCR en utilisant les réseaux GRNN. Nous nous intéressons également à la traduction vocale de ce texte et à son affichage dans une interface simple d'utilisation.

Le premier chapitre présente tout d'abord les notions de base relatives au traitement automatique des caractères. Le chapitre se poursuit par une discussion autour des difficultés liées à l'évaluation des systèmes de traitement automatique de documents. Une conclusion termine le chapitre.

Le deuxième chapitre donne une vue d'ensemble des techniques de classification et apprentissage. Tout en donnant une explication des différentes étapes de la reconnaissance des caractères, de l'utilisation de l'OCR et de la mise en place du réseau GRNN.

Le troisième chapitre contient une description des outils et environnements de travail qui ont permis de réaliser ce projet. Il présente également des informations globales sur la base de données utilisée.

Le quatrième chapitre contient une description détaillée de la méthodologie et de l'implémentation utilisées pour le développement du système intégré, ainsi qu'une présentation des résultats obtenus lors de l'évaluation de ce système.

Finalement, nous partagerons une récapitulation des principales conclusions de la recherche et perspectives d'avenir pour l'amélioration du système intégré, les applications potentielles et les directions de recherche futures.

Chapitre 2

Fondements théoriques et état de l'art dans la reconnaissance de caractères

2.1 Les caractères

Un caractère, dans le contexte de la reconnaissance des caractères, fait référence à une unité de base d'un système d'écriture. C'est un symbole graphique ou une marque qui représente une unité linguistique, telle qu'une lettre, un chiffre, un signe de ponctuation ou un symbole spécial.

A	B	C	D	E	F	G	H	I
J	K	L	M	N	O	P	Q	R
S	T	U	V	W	X	Y	Z	

a	b	c	d	e	f	g	h	i
j	k	l	m	n	o	p	q	r
s	t	u	v	w	x	y	z	

0	1	2	3	4	5	6	7	8
9	.	,	;	:		#	'	!
"		?	%	&	(	)	@	

Plus spécifiquement, voici une définition détaillée d'un caractère :

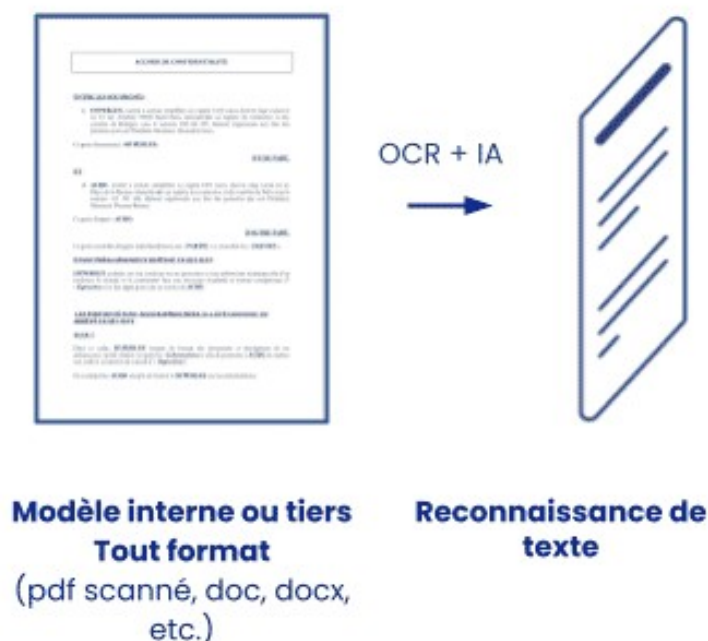
- 1. Unicité :** Chaque caractère est unique et distinct des autres caractères. Il peut être identifié et différencié des autres symboles graphiques.
- 2. Signification :** Chaque caractère a une signification associée dans le système d'écriture utilisé. Par exemple, dans l'alphabet latin, la lettre "A" représente un son spécifique.
- 3. Représentation graphique :** Chaque caractère est représenté par une forme graphique spécifique. Cette forme peut varier en termes de traits, de courbes, de points, de lignes et de leurs arrangements.
- 4. Composants :** Certains caractères peuvent être composés de plusieurs éléments graphiques, appelés traits ou segments. Par exemple, la lettre "A" peut être composée de deux traits diagonaux et d'une ligne horizontale.
- 5. Propriétés typographiques :** Les caractères peuvent avoir des propriétés typographiques telles que la taille, la police, le style (gras, italique, etc.) et la casse (majuscule ou minuscule).
- 6. Contexte et positionnement :** Le sens et l'apparence d'un caractère peuvent varier en fonction de son contexte et de sa position au sein d'un mot, d'une phrase ou d'un texte plus large.
- 7. Reconnaissabilité :** Les caractères doivent être reconnaissables et distinguables par un système de reconnaissance de caractères, qu'il s'agisse d'un être humain ou d'un algorithme informatique.

La reconnaissance des caractères implique l'identification et la classification de ces symboles graphiques pour extraire du sens à partir de textes écrits. Elle joue un rôle crucial dans de nombreuses applications, telles que la numérisation de documents, la traduction automatique, la reconnaissance optique de caractères (OCR), l'indexation de données et bien d'autres.

2.1.1 Reconnaissance automatique des caractères

La reconnaissance automatique des caractères, également connue sous le nom de reconnaissance optique de caractères (OCR), est le processus par lequel les caractères

imprimés ou manuscrits sont convertis en format numérique de manière automatisée. Cette technologie utilise des algorithmes et des techniques de traitement d'images pour analyser les formes et les structures des caractères, afin de les reconnaître et de les transcrire en texte.



La reconnaissance automatique des caractères peut être effectuée en plusieurs étapes. Tout d'abord, l'image contenant les caractères est prétraitée pour améliorer la qualité et l'intelligibilité. Ensuite, les caractères sont segmentés, c'est-à-dire qu'ils sont isolés les uns des autres dans l'image. Ensuite, des techniques de reconnaissance des formes et de l'apprentissage automatique sont utilisées pour identifier les caractères individuels et les associer à des symboles spécifiques. Enfin, les caractères reconnus sont transcrits en texte numérique.

2.1.2 L'objectif de la reconnaissance des caractères

L'objectif principal de la reconnaissance des caractères est de permettre la conversion automatique de texte imprimé ou manuscrit en format numérique. Cela facilite la recherche, l'indexation et la manipulation du texte sur des ordinateurs et d'autres dispositifs électroniques. La reconnaissance des caractères est utilisée dans de nombreux

domaines tels que la numérisation de documents, la transcription automatique, la traduction automatique, la recherche d'informations, la lecture optique de formulaires, etc.

Outre la simple conversion en texte, la reconnaissance des caractères peut également avoir d'autres objectifs, tels que l'analyse de documents pour extraire des informations spécifiques, la détection de mots-clés, la segmentation du texte en unités significatives, la reconnaissance de caractères spécifiques à une langue ou à un domaine particulier, etc. Les objectifs spécifiques déterminent les techniques et les algorithmes utilisés pour la reconnaissance des caractères.

2.1.3 Les problèmes de la reconnaissance de caractères

La reconnaissance de caractères présente plusieurs problèmes et défis. Certains des problèmes courants sont les suivants :

- **Variabilité des formes de caractères** : Les caractères peuvent varier en termes de taille, d'inclinaison, d'épaisseur des traits, de style d'écriture, ce qui rend difficile leur reconnaissance précise.
- **Bruit et dégradation de l'image** : Les caractères peuvent être flous, partiellement effacés, superposés ou perturbés par du bruit dans l'image, ce qui rend la reconnaissance plus difficile.
- **Langues multiples et caractères spéciaux** : La reconnaissance des caractères doit prendre en compte différentes langues, alphabets et systèmes d'écriture, ainsi que des caractères spéciaux tels que les symboles mathématiques, les signes de ponctuation, etc.
- **Écriture manuscrite** : La reconnaissance des caractères manuscrits est plus complexe que celle des caractères imprimés, car les styles d'écriture varient considérablement d'une personne à l'autre, ce qui rend la tâche plus difficile.
- **Gestion des erreurs** : Même avec des techniques avancées, la reconnaissance des caractères peut produire des erreurs. La gestion de ces erreurs et l'amélioration de la précision sont des défis importants.

2.1.4 Domaine d'application

Use Cases



La reconnaissance des caractères est utilisée dans divers domaines et applications, notamment :

- **Numérisation de documents** : La conversion de documents physiques en fichiers numériques permet de stocker, rechercher et manipuler facilement les informations contenues dans ces documents.
- **Transcription automatique** : La reconnaissance des caractères est utilisée pour transcrire automatiquement des enregistrements audio en texte, ce qui facilite la recherche et l'analyse des données.
- **Traduction automatique** : Les systèmes de traduction automatique utilisent souvent la reconnaissance des caractères pour convertir le texte source dans une langue donnée en texte numérique avant de le traduire dans une autre langue.
- **Lecture optique de formulaires** : La reconnaissance des caractères est utilisée pour extraire des informations à partir de formulaires, tels que des questionnaires, des bulletins de vote, des factures, etc.
- **Recherche d'informations** : La reconnaissance des caractères permet d'indexer et de rechercher rapidement des documents numérisés ou des images contenant du texte.
- **Applications mobiles** : La reconnaissance des caractères est utilisée dans les applications mobiles pour la numérisation de cartes de visite, la traduction instantanée, la saisie de texte manuscrit, etc.

Ces applications ne sont que quelques exemples parmi de nombreux autres domaines d'application de la reconnaissance des caractères.

2.2 La traduction de texte

Dans notre projet de reconnaissance de caractères, nous avons pris conscience de l'importance de la traduction pour permettre une utilisation plus large et inclusive de notre application. Pour cela, nous avons choisi d'implémenter la bibliothèque **Google Translate**, qui offre des fonctionnalités de traduction automatique puissantes et fiables.

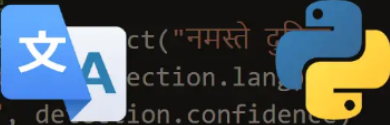
En intégrant la bibliothèque **Google Translate** à notre projet, nous avons pu offrir à nos utilisateurs la possibilité de traduire les caractères détectés dans différentes langues. Cela

permet à notre application de s'adapter aux besoins et aux préférences des utilisateurs de diverses origines linguistiques.

```
# translate a spanish text to arabic for instance
translation = translator.translate("Hola Mundo", dest="ar")
print(f"{translation.origin} ({translation.src}) --> {translation.text}")

# detect a language
detection = translator.detect("नमस्ते दुनिया")
print("Language code: ", detection.lang)
print("Confidence: ", detection.confidence)

# print all available languages
print("Total supported languages: ", len(constants.LANGUAGES))
print("Languages:")
pprint(constants.LANGUAGES)
```



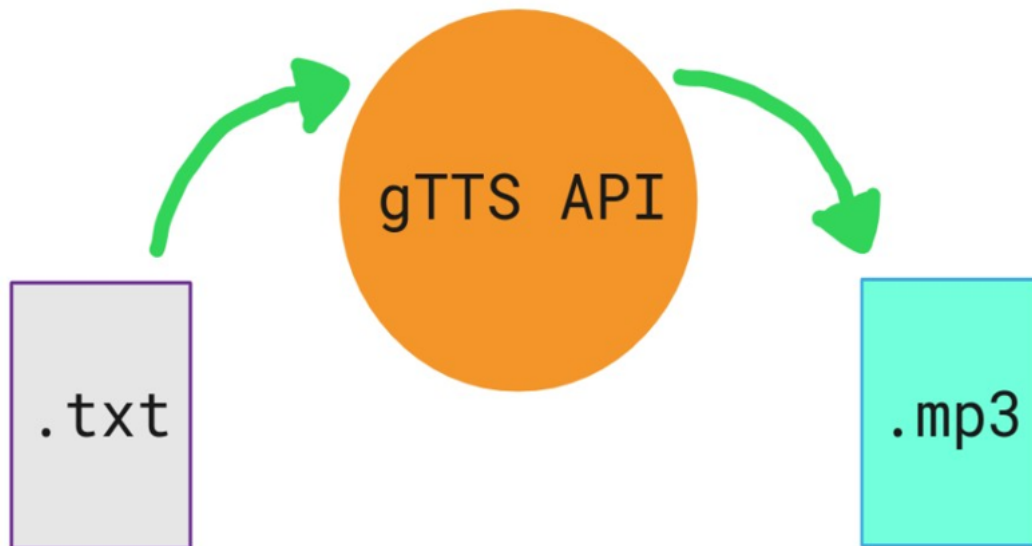
Grâce à la bibliothèque **Google Translate**, notre application est capable de détecter automatiquement la langue source des caractères identifiés et de les traduire efficacement vers la langue cible disponible sélectionnée par l'utilisateur. Cela ouvre de nouvelles opportunités pour la compréhension et la communication, en permettant aux utilisateurs de comprendre et d'interagir avec des textes dans des langues qui leur sont étrangères.

En conclusion, l'intégration de la bibliothèque **Google Translate** dans notre projet de reconnaissance de caractères a renforcé sa valeur en permettant la traduction automatique des caractères détectés. Cette fonctionnalité améliore l'accessibilité et l'utilité de notre application, en facilitant la compréhension et la communication interlinguistique.

2.3 Synthèse vocale

Dans notre projet de reconnaissance de caractères, nous avons décidé d'implémenter de la synthèse vocale pour offrir une expérience plus complète et accessible. Pour cela, nous avons choisi d'intégrer la bibliothèque **gTTS** (Google Text-to-Speech), qui permet de convertir du texte en fichiers audio de synthèse vocale.

En intégrant la bibliothèque **gTTS** à notre projet, nous avons pu donner la possibilité à nos utilisateurs d'entendre la lecture des caractères identifiés. Cela s'avère particulièrement utile pour les utilisateurs ayant une déficience visuelle ou pour ceux qui préfèrent écouter plutôt que lire du texte.

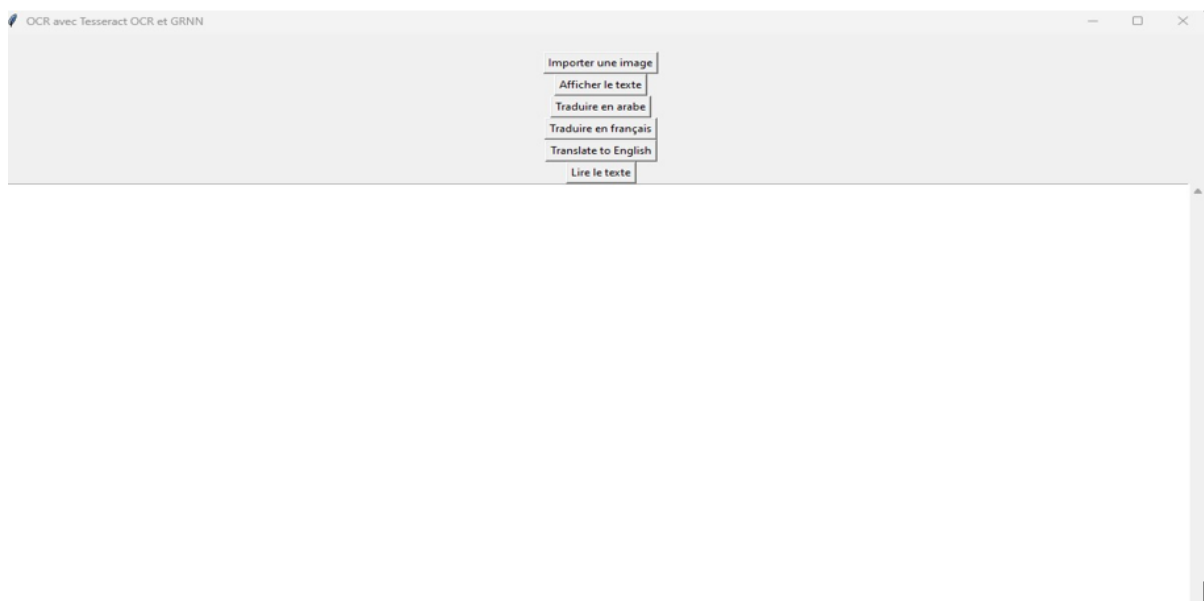


Grâce à la bibliothèque **gTTS**, notre application est en mesure de prendre le texte extrait des caractères et de le convertir en un fichier audio contenant une voix synthétisée. Les utilisateurs peuvent ainsi entendre une lecture claire et précise du texte identifié, facilitant ainsi leur compréhension et leur interaction avec le contenu.

2.4 L'interface utilisateur (GUI)

En dernier, nous avons réalisé une interface utilisateur conviviale pour permettre aux utilisateurs d'interagir facilement avec l'application. Pour cela, nous avons créé une interface utilisateur graphique (GUI) en utilisant la bibliothèque **Tkinter**.

L'objectif de notre interface utilisateur est d'afficher de manière claire et intuitive l'image sélectionnée par l'utilisateur, ainsi que les différentes options d'action disponibles. Pour ce faire, nous avons utilisé les fonctionnalités de mise en page et de gestion des widgets offertes par **Tkinter**.



En utilisant **Tkinter**, nous avons pu afficher l'image sélectionnée dans une zone dédiée, permettant ainsi aux utilisateurs de visualiser facilement le contenu. Nous avons également intégré des boutons d'action correspondant aux fonctionnalités de notre application, tels que l'extraction des caractères et la traduction.

En ce qui concerne le texte extrait/traduit, nous avons utilisé des éléments d'interface utilisateur appropriés, tels que des libellés ou des zones de texte, pour afficher les résultats de manière claire et lisible.

L'interface utilisateur que nous avons créée avec **Tkinter** facilite l'interaction des utilisateurs avec notre application de reconnaissance de caractères. Elle leur permet de sélectionner une image, d'exécuter les actions souhaitées (comme l'extraction des caractères) et de visualiser les résultats sous forme de texte extrait et traduit.

En résumé, la création de l'interface utilisateur avec **Tkinter** a permis d'offrir une expérience conviviale et visuellement attrayante aux utilisateurs de notre application de reconnaissance de caractères. Elle leur permet de naviguer facilement dans les fonctionnalités de l'application, de visualiser les images sélectionnées et les résultats de texte extrait/traduit de manière claire et compréhensible.

Chapitre 3

Approche intégrée basée sur OCR et GRNN

Dans ce chapitre, nous donnerons une vue sur notre méthodologie pour l'utilisation du GRNN et de l'OCR dans la reconnaissance des caractères:

1. Collecte et préparation des données :

- Acquisition d'un ensemble de données contenant des images de caractères imprimés.
- Prétraitement des images pour améliorer la qualité et l'homogénéité des données.

2. Entraînement du modèle OCR :

- Mise en œuvre d'un algorithme OCR approprié pour la reconnaissance des caractères imprimés.
- Entraînement du modèle OCR sur les données prétraitées afin d'apprendre à reconnaître les caractères avec une précision élevée.
- Utilisation de techniques d'apprentissage supervisé pour ajuster les poids et les paramètres du modèle OCR.

3. Mise en place du réseau GRNN :

- Configuration du réseau GRNN en utilisant les caractéristiques appropriées des caractères imprimés.
- Définition des couches du réseau GRNN, y compris les couches d'entrée, les couches récurrentes et les couches de sortie.
- Initialisation des poids et des biais du réseau GRNN.

4. Entraînement du réseau GRNN :

- Entraînement du réseau GRNN sur les données prétraitées pour capturer les motifs et les dépendances séquentielles des caractères.
- Utilisation d'une fonction de perte appropriée et d'un algorithme d'optimisation pour ajuster les paramètres du réseau GRNN.
- Validation croisée pour évaluer les performances du réseau GRNN et éviter le surapprentissage.

5. Intégration de l'OCR et du réseau GRNN :

- Fusion des résultats de l'OCR et du réseau GRNN pour améliorer la précision de la reconnaissance des caractères imprimés.
- Définition de seuils et de stratégies de décision pour combiner les résultats des deux méthodes.

6. Évaluation et validation du système intégré :

- Évaluation de la précision de la reconnaissance des caractères.
- Évaluation de la performance de la traduction de texte.
- Évaluation de l'expérience utilisateur de l'interface conviviale.

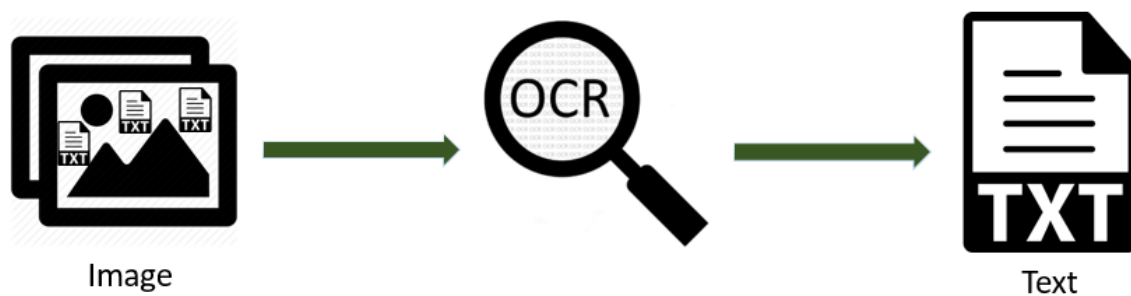
Cette méthodologie permettra de développer et de combiner efficacement l'OCR et le réseau GRNN pour améliorer la reconnaissance des caractères imprimés. Elle comprend également l'évaluation du système intégré pour garantir la qualité des résultats et l'utilisabilité de l'interface conviviale.

3.1 Le modèle OCR

La Reconnaissance Optique de Caractères (OCR) est le processus qui convertit le texte dans une image en codes de caractères modifiables tel que l'ASCII. Par exemple, si vous numérisez un formulaire ou un reçu, votre ordinateur enregistre la numérisation sous forme d'un fichier image. Vous ne pouvez pas utiliser un éditeur de texte pour modifier, rechercher ou compter les mots dans le fichier image. Cependant, vous pouvez utiliser l'OCR pour convertir l'image en un document texte où le contenu est stocké sous forme de données textuelles.

En français, on parle également de Reconnaissance de Caractères (RC) ou de Lecture Automatique de Caractères (LAC). L'OCR utilise des algorithmes et des techniques de traitement d'images pour analyser l'image, reconnaître les caractères et les convertir en

texte. Cela permet de rendre le contenu des documents image accessible et exploitable pour des tâches telles que la recherche, l'analyse de texte, l'extraction de données ou la traduction automatique.



L'OCR peut être utilisé dans de nombreux domaines, tels que la numérisation de documents, la gestion de l'information, l'archivage électronique, la reconnaissance de factures, la lecture de codes-barres, la reconnaissance de plaques d'immatriculation, etc.

3.1.1 La fonctionnalité de l'OCR

Un système de reconnaissance fait généralement appel aux étapes suivantes :

1. Acquisition de l'image

Un scanner lit les documents et les convertit en données binaires. Le logiciel OCR analyse l'image numérisée et classe les zones claires comme arrière-plan et les zones sombres comme du texte.

2. Prétraitement

- Redresser légèrement le document numérisé pour corriger les problèmes d'alignement lors de la numérisation.
- Supprimer les taches d'image numérique ou lisser les contours des images de texte.
- Nettoyer les boîtes et les lignes présentes dans l'image.

3. Reconnaissance de texte

Les deux principaux types d'algorithmes ou de processus logiciels utilisés par un logiciel OCR pour la reconnaissance de texte sont appelés correspondance de motifs (pattern matching) et extraction de caractéristiques (feature extraction).

4. Correspondance de motifs

La correspondance de motifs fonctionne en isolant une image de caractère, appelée glyphe, et en la comparant à un glyphe similaire stocké. La reconnaissance de motifs fonctionne uniquement si le glyphe stocké a une police similaire et une échelle identique au glyphe d'entrée. Cette méthode fonctionne bien avec les images numérisées de documents qui ont été saisis avec une police connue.

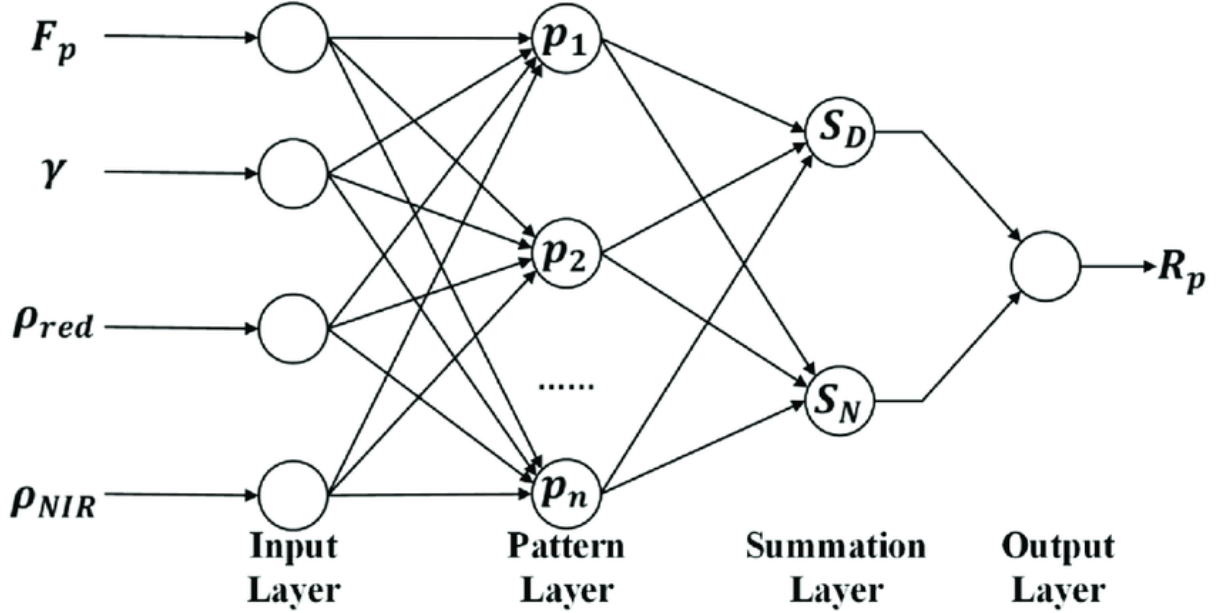
5. Extraction de caractéristiques

L'extraction de caractéristiques décompose ou décompose les glyphes en traits tels que des lignes, des boucles fermées, des directions de ligne et des intersections de lignes. Elle utilise ensuite ces traits pour trouver la meilleure correspondance ou le voisin le plus proche parmi les différents glyphes stockés.

6. Post-traitement

Après l'analyse, le système convertit les données textuelles extraites en un fichier informatique. Certains systèmes OCR peuvent créer des fichiers PDF annotés qui incluent à la fois les versions avant et après du document numérisé.

3.2 Le réseau GRNN



Le fonctionnement détaillé du réseau GRNN est le suivant :

- 1. Couche d'entrée :** Chaque nœud d'entrée reçoit une caractéristique spécifique des données d'entrée. Ces caractéristiques peuvent être continues ou discrètes, et chaque nœud est associé à une valeur d'entrée spécifique.
- 2. Couche des motifs :** La couche des motifs stocke une copie de chaque échantillon d'entraînement. Chaque nœud de cette couche est associé à un échantillon et stocke à la fois ses valeurs d'entrée et sa valeur de sortie correspondante. L'activation de chaque nœud est calculée en utilisant une fonction de similarité, généralement la distance euclidienne, entre le vecteur d'entrée et les échantillons d'entraînement stockés. Les échantillons les plus similaires auront une activation plus élevée.
- 3. Couche de sommation :** La couche de sommation calcule la somme pondérée des sorties de la couche des motifs. Les poids sont déterminés par les distances calculées dans la couche des motifs. Les échantillons d'entraînement les plus similaires ont des poids plus élevés, ce qui signifie qu'ils ont une plus grande influence sur la prédiction finale. Cette couche effectue une agrégation des informations pondérées provenant de la couche des motifs.

4. Couche de sortie : La couche de sortie fournit la prédiction finale du réseau GRNN. Elle est calculée en appliquant une fonction de transfert sur la somme pondérée obtenue à partir de la couche de sommation. Pour les tâches de régression, la sortie est une valeur continue, tandis que pour les tâches de classification, la sortie peut être une valeur discrète ou une probabilité associée à chaque classe.

Dans le contexte de notre recherche, l'utilisation du réseau GRNN peut permettre de capturer les motifs et les dépendances séquentielles des caractères imprimés, améliorant ainsi la précision de la reconnaissance des caractères.

3.2.1 Procédure d'entraînement

La procédure d'entraînement consiste à trouver la valeur optimale de σ . La meilleure pratique consiste à trouver la position où l'erreur quadratique moyenne (MSE) est minimale. Tout d'abord, divisez l'ensemble d'entraînement en deux parties : l'échantillon d'entraînement et l'échantillon de test. Appliquez GRNN aux données de test en fonction des données d'entraînement et calculez le MSE pour différentes valeurs de σ . Ensuite, trouvez le MSE minimum et la valeur correspondante de σ .

3.2.2 Avantages de GRNN

Le principal avantage de GRNN est d'accélérer le processus d'entraînement, ce qui permet au réseau d'être entraîné plus rapidement.

Le réseau est capable d'apprendre à partir des données d'entraînement grâce à un entraînement en "une passe" en une fraction du temps nécessaire pour entraîner les réseaux standard à propagation avant.

La dispersion, σ , est le seul paramètre libre du réseau, qui peut souvent être identifié par la validation croisée en V-fois ou l'échantillonnage fractionné.

Contrairement aux réseaux standard à propagation avant, l'estimation GRNN est toujours capable de converger vers une solution globale et ne sera pas piégée par un minimum local.

En gros, l'avantage du GRNN est qu'il ne nécessite pas d'apprentissage explicite en utilisant un ensemble de données d'entraînement. Au lieu de cela, il apprend au fur et à mesure de son utilisation, en stockant et en utilisant les échantillons d'entraînement pour effectuer des prédictions. Cela permet au GRNN d'être rapide et efficace pour des

tâches de régression, en particulier lorsque les données d'entraînement sont disponibles progressivement au fil du temps.

3.2.3 Inconvénients de GRNN

Sa taille peut être énorme, ce qui rendrait son calcul coûteux.

3.3 Conclusion

La combinaison de l'OCR (Optical Character Recognition) et du GRNN (Gated Recurrent Neural Network) peut être bénéfique pour améliorer la reconnaissance des caractères. Voici quelques raisons possibles pour combiner ces deux approches :

1. Complémentarité des méthodes : L'OCR traditionnel se base sur des techniques de traitement d'image et de vision par ordinateur pour détecter et reconnaître les caractères. Cependant, il peut être limité par des variations d'éclairage, des distorsions ou des problèmes de qualité d'image. Le GRNN, quant à lui, est un type de réseau de neurones récurrent qui peut capturer les dépendances séquentielles dans les données. En combinant ces deux approches, on peut tirer parti des avantages de chacune pour améliorer la performance globale du système de reconnaissance.

2. Adaptabilité aux différentes langues et styles d'écriture : L'OCR traditionnel peut être optimisé pour une langue ou un style d'écriture spécifique, mais il peut avoir des difficultés à reconnaître des caractères provenant d'autres langues ou écritures. Le GRNN est capable d'apprendre des modèles linguistiques et de s'adapter à différents types de caractères, ce qui permet d'améliorer la reconnaissance dans des contextes multilingues ou multistyles.

3. Apprentissage automatique des caractéristiques : L'OCR traditionnel nécessite souvent une extraction manuelle des caractéristiques des images, telles que les contours, les textures ou les histogrammes. Le GRNN, en revanche, peut apprendre automatiquement les caractéristiques pertinentes à partir des données d'entraînement. Cela permet d'éviter le processus laborieux d'extraction manuelle des caractéristiques et peut conduire à de meilleures performances de reconnaissance.

Chapitre 4

Outils et environnements de développement

4.1 Bibliothèques et outils utilisés dans l'implémentation

4.1.1 Python

Python est un langage de programmation interprété et de haut niveau, réputé pour sa simplicité et sa lisibilité. Il a été créé par Guido van Rossum et a été publié pour la première fois en 1991. Python met l'accent sur la lisibilité du code et adopte une philosophie de conception qui favorise une syntaxe claire et concise, ce qui facilite l'écriture et la compréhension du code.



Python prend en charge plusieurs paradigmes de programmation, notamment la programmation procédurale, orientée objet et fonctionnelle. Il dispose d'une vaste bibliothèque standard qui fournit de nombreux modules et fonctions pour effectuer différentes tâches, ce qui le rend très polyvalent.

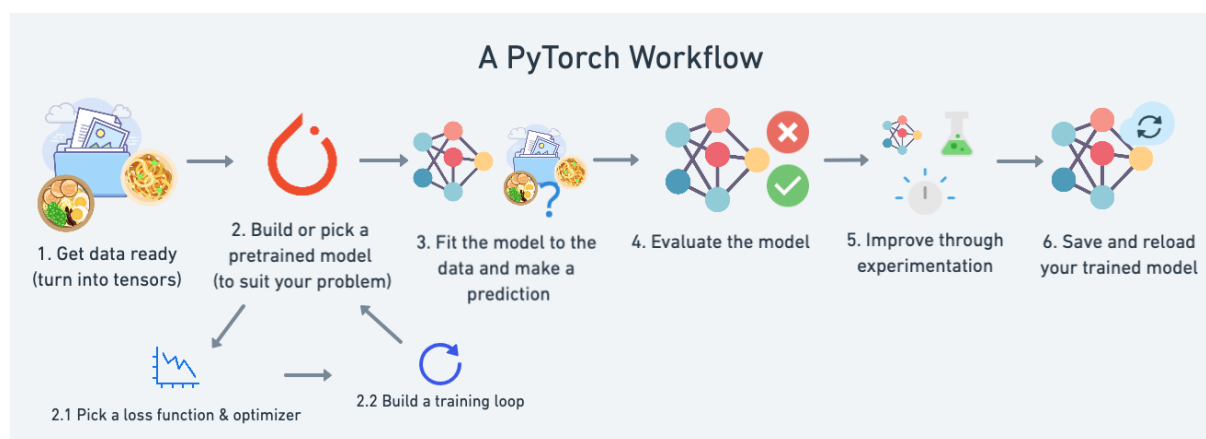
L'une des caractéristiques clés de Python est l'utilisation de l'indentation pour définir les blocs de code, ce qui favorise la lisibilité du code et encourage les bonnes pratiques de programmation. Python est largement utilisé dans de nombreux domaines, tels que le développement web, l'analyse de données, l'intelligence artificielle, l'automatisation des tâches et bien d'autres encore.

Sa syntaxe simple et expressive, sa grande communauté de développeurs et sa facilité d'apprentissage en ont fait l'un des langages de programmation les plus populaires et les plus appréciés par les programmeurs du monde entier.

4.1.2 PYTORCH



PyTorch est un framework complet pour la construction de modèles d'apprentissage profond, qui est un type d'apprentissage automatique couramment utilisé dans des applications telles que la reconnaissance d'images et le traitement du langage. Écrit en Python, il est relativement facile pour la plupart des développeurs en apprentissage automatique de l'apprendre et de l'utiliser. PyTorch se distingue par son excellent support des GPU et son utilisation de l'autodifférenciation en mode inverse, ce qui permet de modifier les graphiques de calcul en temps réel. Cela en fait un choix populaire pour une expérimentation et un prototypage rapide.



Principaux avantages de PyTorch :

- Il est écrit en Python et intégré aux bibliothèques Python populaires telles que NumPy pour le calcul scientifique, SciPy et Cython pour la compilation de Python en C afin d'obtenir de meilleures performances.
- PyTorch prend en charge les graphiques de calcul dynamiques, ce qui permet de modifier le comportement du réseau à l'exécution. Cela offre un avantage de flexibilité majeur par rapport à la majorité des frameworks d'apprentissage automatique, qui nécessitent que les réseaux neuronaux soient définis comme des objets statiques avant l'exécution.
- Le Hub PyTorch est un référentiel de modèles pré-entraînés qui peuvent être utilisés, dans certains cas, avec une seule ligne de code.
- De nouveaux composants personnalisés peuvent être créés en tant que sous-classes d'une classe Python standard, les paramètres peuvent être facilement partagés avec des outils externes tels que TensorBoard, et des bibliothèques peuvent être facilement importées et utilisées en ligne.
- PyTorch dispose d'un ensemble d'API bien considéré qui peut être utilisé pour étendre les fonctionnalités de base.
- Il dispose d'une vaste collection d'outils et de bibliothèques dans des domaines allant de la vision par ordinateur à l'apprentissage par renforcement.
- PyTorch et TensorFlow se ressemblent en ce sens que les composants principaux des deux sont les tenseurs et les graphiques.

Les tenseurs sont des structures de données multidimensionnelles, similaires aux tableaux, qui sont utilisées pour représenter les données dans les frameworks de deep learning. Les deux frameworks offrent des fonctionnalités pour créer, manipuler et effectuer des opérations sur des tenseurs.

Les graphiques, quant à eux, sont des structures qui représentent les opérations et les calculs effectués sur les tenseurs. Les frameworks utilisent des graphiques pour définir et organiser les opérations à exécuter, ce qui permet d'optimiser les calculs et d'exploiter efficacement les ressources matérielles, telles que les GPU.

Dans PyTorch, les graphiques sont définis de manière dynamique, ce qui signifie que les opérations sont exécutées au fur et à mesure qu'elles sont définies, ce qui facilite l'expérimentation et le débogage.

4.1.3 NumPy



NumPy est une bibliothèque Python largement utilisée pour le calcul scientifique et numérique. Son nom est l'abréviation de "Numerical Python". NumPy fournit des structures de données de tableau multidimensionnel, des fonctions mathématiques de haut niveau et des outils pour effectuer des opérations efficaces sur ces tableaux.

La principale structure de données fournie par NumPy est appelée "ndarray" (tableau multidimensionnel). Les ndarray permettent de stocker et de manipuler des données de manière efficace, ce qui en fait une base essentielle pour de nombreuses tâches scientifiques et analytiques.

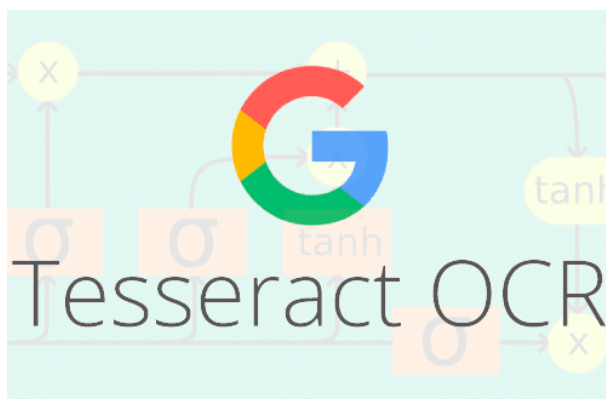
NumPy offre une large gamme de fonctions mathématiques, telles que les opérations de base (addition, soustraction, multiplication, division), les fonctions trigonométriques, les fonctions exponentielles et logarithmiques, les opérations matricielles, et bien plus encore.

Ces fonctions sont optimisées pour des performances élevées et permettent de réaliser des calculs numériques de manière efficace.

Une autre caractéristique clé de NumPy est sa capacité à effectuer des opérations vectorisées. Cela signifie qu'au lieu d'itérer sur chaque élément d'un tableau, les opérations sont appliquées à l'ensemble du tableau en une seule fois, ce qui améliore les performances et simplifie le code.

En résumé, NumPy est une bibliothèque Python essentielle pour le calcul scientifique et numérique. Elle fournit des structures de données efficaces, des fonctions mathématiques avancées et des opérations vectorisées, ce qui en fait un outil puissant pour le traitement et l'analyse des données numériques.

4.1.4 Python-tesseract



Python-tesseract est un outil de reconnaissance optique de caractères (OCR) pour Python qui est capable de reconnaître et de "lire" le texte contenu dans les images.

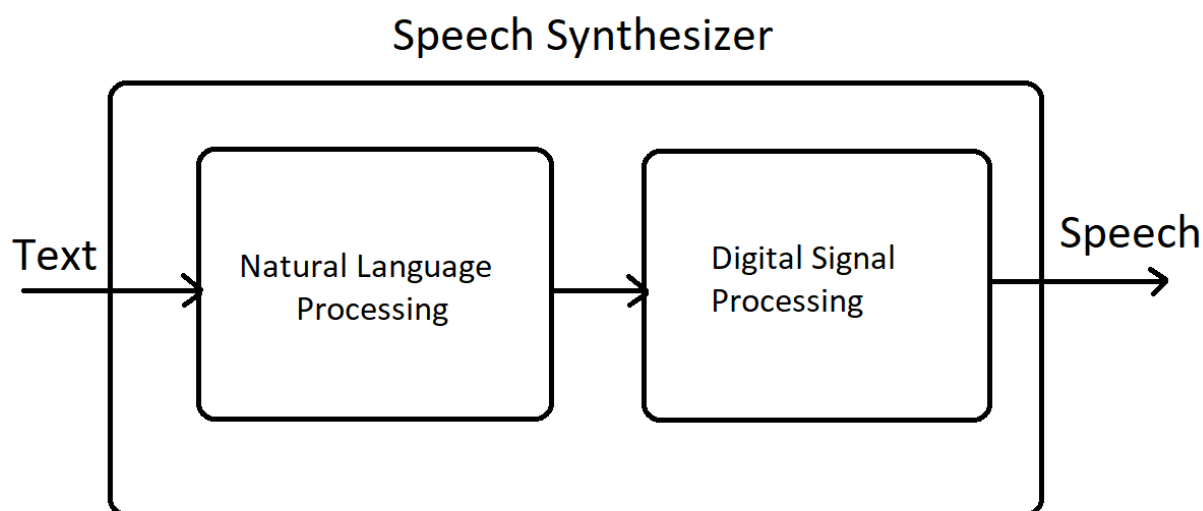
L'OCR est une technologie qui permet d'extraire le texte à partir d'images ou de documents scannés. Python-tesseract est une bibliothèque Python qui s'appuie sur la bibliothèque Tesseract-OCR, un moteur OCR open source très populaire.

Avec Python-tesseract, vous pouvez importer des images dans votre code Python et utiliser la fonctionnalité OCR pour extraire le texte de ces images. Cela peut être utile dans de nombreuses applications, telles que la numérisation de documents, la reconnaissance de factures, la lecture de codes-barres ou la conversion de documents imprimés en texte éditable.

Python-tesseract offre une interface simple et facile à utiliser. Il prend en charge différents formats d'images tels que JPEG, PNG, TIFF, etc. Une fois le texte extrait, vous pouvez le traiter et l'utiliser dans votre application selon vos besoins.

Il convient de noter que la précision de la reconnaissance dépend de la qualité de l'image et de la lisibilité du texte. Il peut être nécessaire de prétraiter les images, par exemple en ajustant le contraste ou en redimensionnant l'image, pour obtenir de meilleurs résultats de reconnaissance.

4.1.5 gTTS (Google Text-to-Speech)

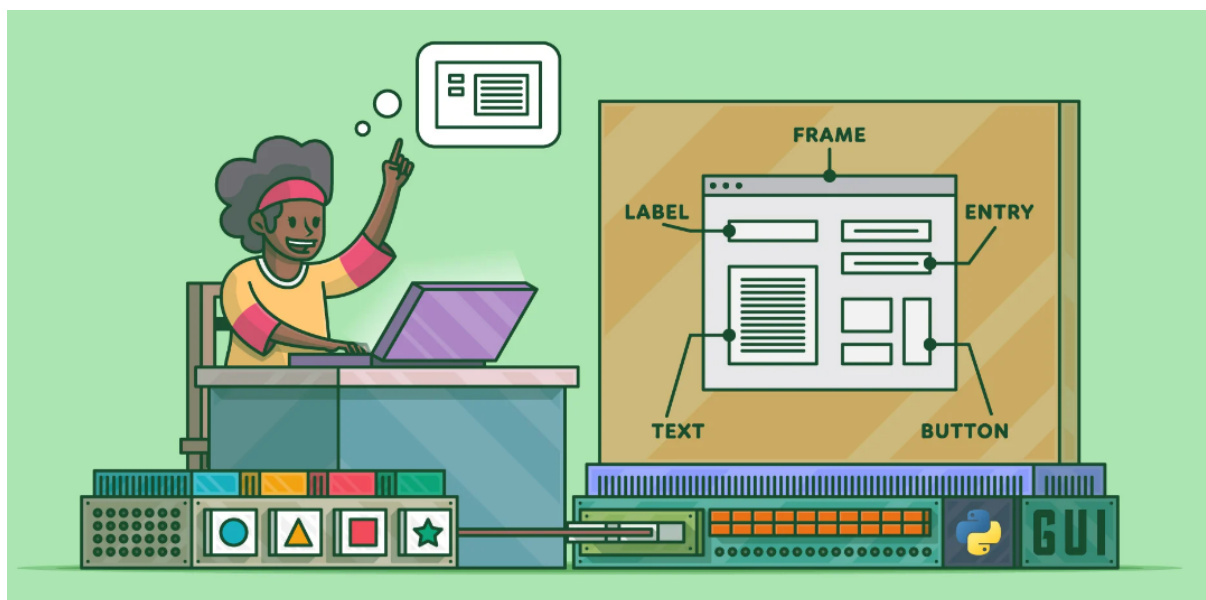


gTTS (Google Text-to-Speech) est une bibliothèque Python qui permet de convertir du texte en parole en utilisant les services de synthèse vocale de Google. gTTS est un outil pratique pour générer de la parole à partir de texte en français et dans d'autres langues supportées par Google. Vous pouvez également ajuster des paramètres tels que la vitesse de lecture, le volume, ou ajouter des pauses en utilisant les options disponibles dans la bibliothèque.

4.1.6 pyttsx3

pyttsx3 est une bibliothèque de conversion de texte en discours en Python qui utilise le moteur de synthèse vocale du système d'exploitation sur lequel il est exécuté, ce qui lui permet de fonctionner hors ligne. Vous pouvez également personnaliser les voix, les vitesses de parole et d'autres paramètres en utilisant les options disponibles dans la bibliothèque.

4.1.7 Tkinter (Tool kit interface)



Tkinter (Tool kit interface) est la bibliothèque graphique libre d'origine pour le langage Python, permettant la création d'interfaces graphiques. Tkinter offre de nombreux autres widgets tels que des boutons, des champs de texte, des listes déroulantes, des cases à cocher, etc., ainsi que des fonctionnalités pour gérer les événements, les mises en page et les styles graphiques. Bien que Tkinter soit considérée comme une bibliothèque graphique de base, elle offre néanmoins des fonctionnalités suffisantes pour créer des interfaces graphiques fonctionnelles. Pour des besoins plus avancés.

4.2 Base de données MNIST

La base de données MNIST (Modified National Institute of Standards and Technology database) est une vaste collection d'images de chiffres manuscrits, largement utilisée pour entraîner et évaluer des systèmes de traitement d'images et des modèles d'apprentissage automatique. Elle est largement considérée comme un ensemble de données standard dans le domaine de la vision par ordinateur et de l'apprentissage automatique.

MNIST contient des images en niveaux de gris de petites dimensions de 28x28 pixels représentant des chiffres manuscrits allant de 0 à 9. Elle contient un total de 70 000 images de chiffres écrits à la main, avec un ensemble d'entraînement comprenant 60 000 images et un ensemble de test comprenant 10 000 images. Toutes les images sont étiquetées avec le chiffre correspondant qu'elles représentent. Il y a un total de 10 classes de chiffres (de 0 à 9).

Chapitre 5

Application et implémentation pratique

Voici les étapes générales par lesquelles une image passe dans un système de reconnaissance de caractères basé à la fois sur le GRNN et l'OCR, permettant ainsi la traduction du texte et la synthèse vocale :

- 1. Acquisition de l'image :** Tout d'abord, l'image contenant le texte à reconnaître est acquise et importée.
- 2. Prétraitement de l'image :** L'image est prétraitée pour améliorer sa qualité et faciliter la reconnaissance des caractères. Cela peut inclure des étapes telles que le redimensionnement, la correction de la perspective, l'amélioration du contraste et de la luminosité, ainsi que la réduction du bruit.
- 3. Détection des régions de texte :** À l'aide de techniques d'OCR combinées à un réseau de neurones GRNN, les régions de l'image susceptibles de contenir du texte sont détectées. Cela peut se faire en utilisant des algorithmes de détection de contours, de classification ou de segmentation d'objets.
- 4. Extraction des caractères :** Les régions de texte détectées sont segmentées en caractères individuels. Cela peut impliquer la séparation des caractères à partir des espaces blancs, la suppression des éléments indésirables ou la fusion des caractères connectés.
- 5. Reconnaissance des caractères :** Les caractères extraits sont soumis à l'OCR pour la reconnaissance proprement dite. L'OCR est combiné à un réseau de neurones GRNN (General Recurrent Neural Network), pour reconnaître les caractères individuels.
- 6. Traduction du texte :** Une fois les caractères reconnus, le texte résultant est traduit

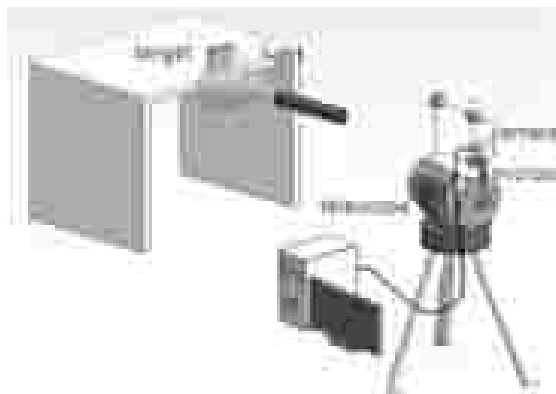
dans la langue souhaitée à l'aide d'outils de traduction automatique implémenté à l'aide de bibliothèques dédiées.

7. Synthèse vocale : Le texte traduit est ensuite utilisé en entrée d'un moteur de synthèse vocale qui convertit le texte en parole.

5.1 Acquisition de l'image

L'acquisition d'images dans le traitement d'images peut être largement définie comme l'action de récupérer une image à partir d'une source, généralement une source matérielle, afin qu'elle puisse être transmise à tous les processus qui doivent se produire par la suite. L'acquisition d'images dans le traitement d'images est toujours la première étape de la séquence de workflow car, sans image, aucun traitement n'est possible.

L'image qui est acquise est complètement non traitée et est le résultat du matériel utilisé pour la générer, ce qui peut être très important dans certains domaines pour avoir une base de référence cohérente à partir de laquelle travailler. L'un des objectifs ultimes de ce processus est d'avoir une source d'entrée qui fonctionne selon des directives contrôlées et mesurées telles que la même image peut, si nécessaire, être presque parfaitement reproduite dans les mêmes conditions afin que les facteurs anormaux soient plus faciles à localiser et à éliminer.



5.2 Prétraitement de l'image

La manipulation des données consiste à partitionner la base de données utilisée entre les données de test et d'entraînement, les techniques utilisées sont principalement basées sur

le fait que nous avons considéré pour le prétraitement la bibliothèque Pytorch.

Dans le contexte du prétraitement des données sur la base de données MNIST, les modules importés de la bibliothèque torch jouent les rôles suivants :

torch.nn: Ce module fournit les classes de base pour définir et former des réseaux de neurones. Il contient des classes telles que Module (la classe de base pour tous les modules de réseau de neurones), ainsi que différentes couches (comme les couches linéaires, les couches de convolution, etc.) utilisées pour construire les réseaux.

torchvision.datasets: Ce module fournit des fonctions et des classes pour charger et manipuler des ensembles de données couramment utilisés dans la vision par ordinateur. Dans ce cas, on utilise `dsets.MNIST` pour charger l'ensemble de données MNIST, qui est un ensemble de données d'images de chiffres manuscrits.

torchvision.transforms: Ce module contient des fonctions de transformation d'images qui sont appliquées aux données. Dans ce cas, on utilise `transforms.ToTensor()` pour convertir les images PIL en tenseurs torch, ce qui est une représentation couramment utilisée pour les calculs dans PyTorch.

torch.autograd.Variable: Ce module fournit la classe Variable, qui est utilisée pour encapsuler les tenseurs torch et enregistrer automatiquement les opérations effectuées sur eux pour le calcul du gradient. Il est utilisé pour permettre la rétropropagation du gradient et la mise à jour des poids du réseau pendant l'entraînement.

La bibliothèque torch et ses modules fournissent les fonctionnalités nécessaires pour construire et former des réseaux de neurones, charger et transformer des ensembles de données, ainsi que pour effectuer les calculs et les opérations de rétropropagation nécessaires pour l'entraînement des modèles sur la base de données MNIST.

5.3 Les paramètres principaux pour l'entraînement

5.3.1 `batch_size` et `num_epochs`

Le `batch_size` et `num_epochs` sont des hyperparamètres utilisés lors de l'entraînement d'un modèle d'apprentissage automatique.

Batch Size (Taille du lot) : fait référence au nombre d'échantillons d'apprentissage qui sont propagés à travers le réseau de neurones avant que les poids du modèle ne soient mis à jour lors de la rétropropagation. Il spécifie combien d'exemples de données sont utilisés simultanément pour calculer les gradients et effectuer une mise à jour des poids.

Le choix d'une taille de lot appropriée est important pour l'efficacité et la précision de l'apprentissage. Un `batch_size` trop petit peut entraîner une convergence lente du modèle et peut nécessiter plus d'itérations pour atteindre de bonnes performances.

Un **`batch_size`** trop grand peut nécessiter plus de mémoire et peut ralentir le processus d'apprentissage. Il est souvent recommandé de choisir une taille de lot qui tient compte des ressources matérielles disponibles et qui offre un bon compromis entre efficacité et précision.

Number of Epochs (Nombre d'époques): fait référence au nombre total de fois où l'ensemble de données complet est passé par le modèle lors de l'entraînement. Une époque correspond à une itération complète sur tous les exemples de données d'entraînement.

L'entraînement d'un modèle se fait généralement en itérant sur les données pendant plusieurs époques. Pendant chaque époque, le modèle utilise les données d'entraînement pour effectuer une rétropropagation, mettre à jour les poids et ajuster ses performances. Un nombre plus élevé d'époques permet au modèle de voir les exemples de données à plusieurs reprises et d'apprendre des motifs plus complexes.

Cependant, il est important de noter qu'un nombre d'époques trop élevé peut entraîner un surapprentissage (overfitting), où le modèle mémorise les données d'entraînement au lieu de généraliser les motifs. Le choix du nombre optimal d'époques dépend du problème, de la taille de l'ensemble de données et de la complexité du modèle.

En résumé, le `batch_size` détermine la quantité d'exemples de données utilisés pour calculer les gradients et mettre à jour les poids du modèle à chaque itération, tandis que le `num_epochs` spécifie le nombre total d'itérations complètes sur l'ensemble de données d'entraînement. Ces hyperparamètres jouent un rôle crucial dans la formation d'un modèle d'apprentissage automatique.

5.3.2 Mélange des données : shuffle

Le paramètre **shuffle** (**mélange**) est utilisé dans le contexte des `DataLoaders` pour spécifier si les données doivent être mélangées ou non avant chaque époque lors de l'entraînement d'un modèle d'apprentissage automatique.

Lorsque `shuffle` est défini sur `True`, les données sont mélangées aléatoirement avant chaque époque. Cela signifie que l'ordre dans lequel les exemples de données sont présentés au modèle lors de chaque époque est aléatoire. Le mélange des données est souvent recommandé car il permet d'introduire de l'aléa dans l'ordre de présentation des exemples de données, ce qui peut aider à prévenir les biais liés à l'ordre des données d'entraînement.

Le mélange des données est particulièrement important lorsque les données sont organisées de manière séquentielle, par exemple, lorsque les classes sont regroupées en blocs consécutifs. Si les données ne sont pas mélangées et que l'ordre des classes est maintenu, cela peut conduire à un modèle qui s'adapte davantage à l'ordre des classes qu'aux caractéristiques intrinsèques des données.

En revanche, lorsque `shuffle` est défini sur `False`, les données sont présentées au modèle dans l'ordre dans lequel elles sont stockées. Cela peut être utile, par exemple, lors de l'évaluation du modèle sur un ensemble de données de test où l'ordre n'a pas d'importance.

5.4 Le concept POO sur python

Le concept de programmation orientée objet (POO) est une approche de programmation qui se concentre sur la création de classes et d'objets. Python est un langage qui prend en charge la programmation orientée objet et offre des fonctionnalités puissantes pour la mise en œuvre de ce paradigme.

5.4.1 Classes

Les classes sont des modèles qui définissent la structure et le comportement des objets. Elles sont créées à l'aide du mot-clé `class`. Les classes sont utilisées pour regrouper des attributs (variables) et des méthodes (fonctions) associés.

5.4.2 Objets

Les objets sont des instances d'une classe spécifique. Ils représentent des entités du monde réel ou des concepts abstraits. Un objet possède des attributs qui sont des variables spécifiques à cet objet et des méthodes qui sont des fonctions spécifiques à cet objet. Les objets sont créés en utilisant le constructeur de classe.

5.4.3 Attributs

Les attributs sont des variables qui décrivent les caractéristiques d'un objet. Ils peuvent être des variables d'instance, qui sont spécifiques à chaque objet, ou des variables de classe, qui sont partagées par tous les objets d'une classe. Les attributs sont définis dans la classe et sont accessibles à travers les objets en utilisant la notation dot (objet.attribut).

5.4.4 Methodes

Les méthodes sont des fonctions associées à une classe. Elles définissent le comportement des objets de cette classe. Les méthodes peuvent être des méthodes d'instance, qui agissent sur un objet spécifique, ou des méthodes de classe, qui agissent sur la classe elle-même. Les méthodes sont définies dans la classe et sont appelées en utilisant la notation dot (objet.methode()).

5.4.5 Encapsulation

La POO encourage l'encapsulation, qui consiste à regrouper des données (attributs) et des opérations (méthodes) associées dans une classe. L'encapsulation permet de cacher les détails internes d'une classe à l'extérieur et de fournir une interface claire pour interagir avec les objets.

5.4.6 Héritage

L'héritage permet de créer de nouvelles classes en utilisant les fonctionnalités d'une classe existante. Une classe héritante (ou sous-classe) hérite des attributs et des méthodes de

la classe parente (ou superclasse) et peut également ajouter de nouvelles fonctionnalités ou modifier le comportement existant. L'héritage permet de réutiliser du code et de créer une hiérarchie de classes.

5.4.7 polymorphisme

Le polymorphisme permet à des objets de différentes classes d'être traités de manière similaire. Cela signifie qu'un objet peut être utilisé de différentes manières en fonction de son type ou de sa classe. Le polymorphisme est réalisé grâce à l'héritage et au concept de méthodes polymorphes.

La programmation orientée objet en Python offre une approche puissante pour structurer et organiser votre code, améliorer la réutilisabilité et la modularité, et représenter des concepts du monde réel de manière plus intuitive. La connaissance de ces concepts fondamentaux de la POO en Python est essentielle pour développer des applications robustes.

5.5 Reconnaissance et classification des caractères

5.5.1 La construction du modèle GRNN

Le modèle GRNN est basé sur le concept de la porte de mise à jour (update gate) et de la porte de réinitialisation (reset gate). Ces portes permettent au modèle de contrôler l'information qui est propagée à travers les étapes temporelles du réseau.

5.5.1.1 Le fonctionnement du modèle GRNN

1. Initialisation : Le modèle GRNN est initialement configuré avec des poids aléatoires. Les paramètres du modèle incluent la taille de l'entrée (`input_size`), la taille cachée (`hidden_size`) et le nombre de couches (`num_layers`).
2. Passage d'entrée : À chaque étape temporelle, le modèle GRNN reçoit une entrée (séquence) de taille `input_size`. Cette entrée est propagée à travers le réseau récurrent.

3. Portes de réinitialisation et de mise à jour : À chaque étape temporelle, le modèle calcule les valeurs des portes de réinitialisation (reset gate) et de mise à jour (update gate). Ces portes sont des vecteurs de la même taille que la taille cachée (hidden_size) et elles déterminent quelles informations seront conservées et quelles informations seront oubliées.
4. Calcul de l'état caché : Le modèle calcule l'état caché (hidden state) en utilisant les portes de réinitialisation et de mise à jour ainsi que l'entrée actuelle et l'état caché précédent. Cela permet au modèle de mettre à jour et de mémoriser les informations pertinentes pour la tâche en cours.
5. Prédiction de sortie : En fonction de l'état caché, le modèle peut effectuer une prédiction ou générer une sortie correspondante à l'étape temporelle actuelle. La taille de la sortie dépend de la tâche spécifique pour laquelle le modèle GRNN est utilisé.
6. Répétition : Les étapes 2 à 5 sont répétées pour chaque étape temporelle de la séquence.

Le modèle GRNN est généralement entraîné en utilisant des techniques d'apprentissage supervisé telles que la rétropropagation du gradient (backpropagation) pour ajuster les poids du réseau afin de minimiser l'erreur entre les prédictions du modèle et les valeurs réelles.

5.5.1.2 Les optimiseurs

La fonction de perte **CrossEntropyLoss()** est couramment utilisée dans les tâches de classification multiclasse lors de l'entraînement de réseaux de neurones. Elle est particulièrement adaptée lorsque les classes sont mutuellement exclusives, c'est-à-dire qu'un exemple de données ne peut appartenir qu'à une seule classe.

1. Entrées de la fonction : La fonction **CrossEntropyLoss()** prend deux entrées : les prédictions du modèle et les étiquettes (labels) correspondantes. Les prédictions du modèle sont généralement les scores ou les probabilités prédites pour chaque classe, tandis que les étiquettes représentent les classes réelles des exemples de données.

2. Conversion des prédictions en probabilités : Avant de calculer la perte, les prédictions du modèle sont généralement converties en probabilités en utilisant une fonction d'activation appropriée, comme la fonction softmax. Cela garantit que les prédictions sont des valeurs comprises entre 0 et 1 et que leur somme est égale à 1, ce qui représente une distribution de probabilités sur les classes.
3. Calcul de la perte : La fonction `CrossEntropyLoss()` calcule la perte en comparant les probabilités prédites par le modèle aux étiquettes réelles. Elle mesure la divergence entre les distributions de probabilités. La formule de la perte Cross Entropy est :

$$L(\mathbf{p}, \mathbf{q}) = - \sum (p_i \cdot \log(q_i))$$

où p_i est la probabilité réelle de la classe i et q_i est la probabilité prédite de la classe i .

4. Agrégation de la perte : La perte individuelle pour chaque exemple de données est calculée, puis agrégée en une seule valeur de perte en utilisant une opération d'agrégation, comme la moyenne ou la somme.
5. Rétropropagation du gradient : Une fois que la perte est calculée, la rétropropagation du gradient est utilisée pour propager l'erreur à travers le réseau de neurones et ajuster les poids afin de minimiser la perte. Cela permet d'améliorer les performances du modèle au fil de l'entraînement.

La fonction de perte **CrossEntropyLoss()** est largement utilisée car elle est capable de gérer efficacement des problèmes de classification multiclasse. Elle pénalise davantage les prédictions incorrectes en attribuant une perte plus élevée lorsque la probabilité prédite pour la classe correcte est faible. En conséquence, elle encourage le modèle à apprendre des représentations discriminantes pour chaque classe et à améliorer sa capacité de classification.

Il est important de noter que la fonction **CrossEntropyLoss()** est généralement utilisée en combinaison avec une fonction d'activation softmax sur les sorties du modèle pour obtenir des probabilités normalisées.

5.6 Traduction du texte

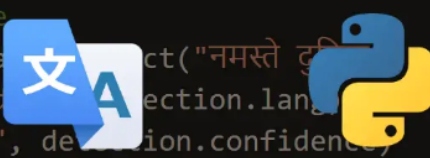
La traduction consiste à transposer un texte écrit d'une langue à une autre, en transmettant le plus fidèlement possible le message. Le traducteur traduit généralement d'une 2e ou d'une 3e langue vers sa langue maternelle. Il est de nature curieuse, a une vaste culture, une grande souplesse d'esprit, une très bonne connaissance de ses langues de travail et des aptitudes à rédiger. La discipline se distingue de l'interprétation, qui consiste à reformuler oralement d'une langue à une autre un message lors de discours, de réunions, de conférences et de débats, ou encore devant des cours de justice ou des tribunaux administratifs.

En python, la méthode `translate()` renvoie une copie d'une chaîne dans laquelle tous les caractères ont été traduits à l'aide de la table (construite avec la fonction `maketrans()` dans le module `String`), en supprimant éventuellement tous les caractères trouvés dans la chaîne donnée.

```
# translate a spanish text to arabic for instance
translation = translator.translate("Hola Mundo", dest="ar")
print(f"{translation.origin} ({translation.src}) --> {translation.text}")

# detect a language
detection = translator.detect("नमस्ते दुनिया")
print("Language code: ", detection.language)
print("Confidence: ", detection.confidence)

# print all available languages
print("Total supported languages:", len(constants.LANGUAGES))
print("Languages:")
pprint(constants.LANGUAGES)
```



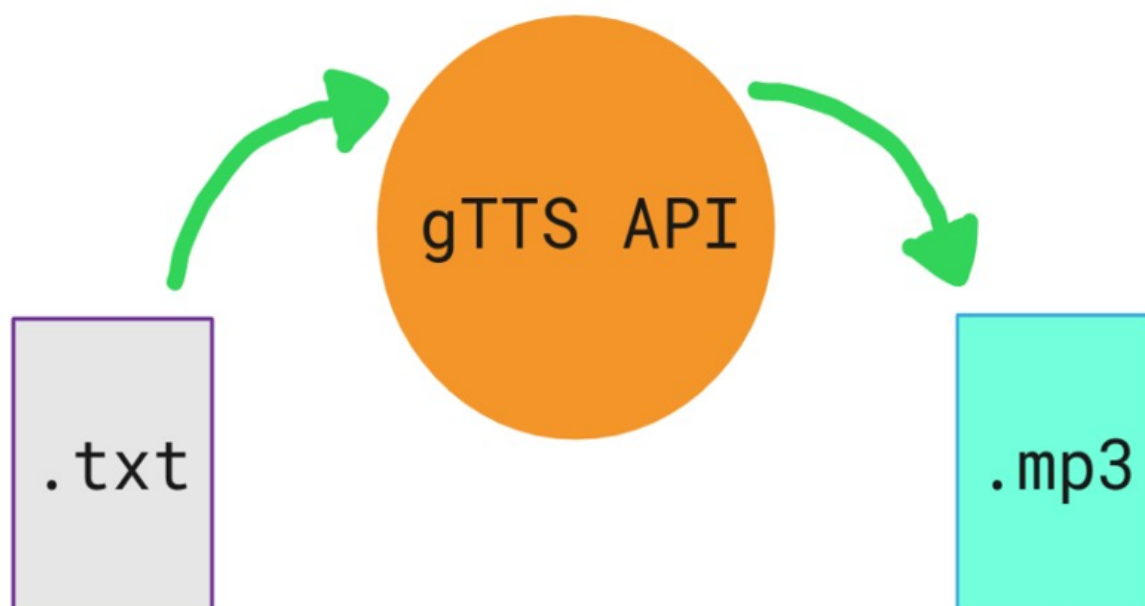
5.7 Synthèse vocale

La synthèse vocale permet de transformer un texte en une suite de phonèmes en essayant de se rapprocher le plus possible de la parole humaine.

Ce dispositif utilisable à partir de nombreux appareils (ordinateur, tablette ou smartphone) génère à l'aide d'un logiciel une voix synthétique qui va pouvoir traduire un texte numérique.

En python, pyttsx3 est une bibliothèque Python qui nous permet de convertir du texte en parole. Nous lui fournirons donc notre texte et il convertira ce texte en audio.

Il s'agit d'un wrapper autour de plusieurs moteurs de synthèse vocale, y compris le moteur Text-to-Speech (TTS) de Microsoft.

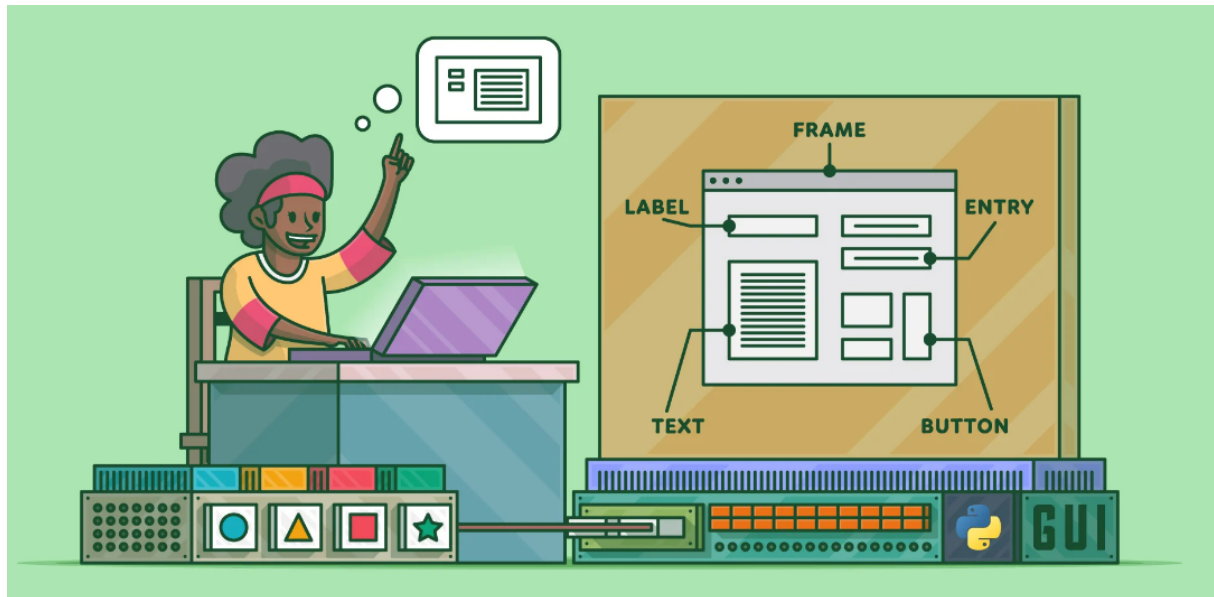


5.8 Création de l'interface

L'interface graphique désigne la manière dont est présenté un logiciel à l'écran pour l'utilisateur. C'est le positionnement des éléments : menus, boutons, fonctionnalités dans la fenêtre. Une interface graphique bien conçue est ergonomique et intuitive afin que l'utilisateur la comprenne tout de suite.

Python inclut une interface orientée objet pour le jeu d'objets graphiques Tcl/Tk, appelée tkinter. C'est probablement la plus facile à installer et à utiliser, car elle est incluse avec

la plupart des distributions binaires de Python¹. Pour créer une interface graphique avec Tkinter, il faut créer une fenêtre racine et lancer la boucle principale via la méthode `mainloop()`. L'appel à cette méthode bloque l'exécution de l'appelant. Tkinter est alors à l'écoute des événements provenant de l'utilisateur, tels que des clics de souris².



Chapitre 6

Conclusion

Nous avons proposé, à travers ce projet, le développement d'un système intégré de reconnaissance de caractères imprimés, de traduction et synthèse vocale de texte à travers une interface conviviale, offrant une expérience utilisateur fluide et efficace. Les avancées réalisées dans cette recherche ont abordé plusieurs problématiques essentielles.

Tout d'abord, des techniques avancées de reconnaissance de caractères ont été explorées, en mettant l'accent sur l'extraction robuste des caractéristiques et leur classification. L'utilisation de l'OCR et des réseaux GRNN a permis d'améliorer la précision et la robustesse de la reconnaissance des caractères imprimés.

Ensuite, le système a été étendu pour intégrer une fonction de traduction automatique du texte reconnu dans différentes langues. Cette fonctionnalité permet aux utilisateurs d'obtenir rapidement une traduction vocale du texte imprimé, facilitant ainsi la communication multilingue et la compréhension dans des contextes différents.

En plus des avancées mentionnées précédemment, nous avons également réalisé une synthèse vocale du texte reconnu. La synthèse vocale consiste à convertir le texte en parole, offrant ainsi une expérience encore plus immersive et accessible.

Enfin, une interface a été développée pour faciliter l'interaction entre les utilisateurs et le système. Cette interface intuitive permet l'importation d'images contenant du texte, la visualisation des résultats de reconnaissance, la traduction et la synthèse vocale instantanée.

Ce système intégré de reconnaissance de caractères présente de nombreuses applications potentielles, notamment dans les domaines de la numérisation de documents multilingues et de la reconnaissance des plaques d'immatriculation.

En conclusion, nous avons réussi à développer un système complet et polyvalent répondant aux problématiques de reconnaissance de caractères imprimés. Les avancées réalisées dans cette recherche ouvrent de nouvelles perspectives pour la transformation numérique des documents, la communication multilingue et l'accessibilité linguistique. Des possibilités d'améliorations futures et de nouvelles recherches sont également envisagées pour poursuivre le développement de ce système intégré.

Références

- [1] spiegato *Acquisition*.; <https://spiegato.com/fr/quest-ce-que-lacquisition-dimages-dans-le-traitement-dimages>
- [2] ling-trad *Traduction de texte*.; <https://ling-trad.umontreal.ca/departement/quest-ce-que-la-traduction>
- [3] primabord *Synthèse vocale*.; <https://primabord.eduscol.education.fr/qu-est-ce-que-la-synthese-vocale>
- [4] bing *Interface graphique*.; <https://www.bing.com/interface+graphique+in+python>